

Efficient visualization of appearance space of translucent objects using differential rendering

Riel Suzuki
Hokkaido University
Japan
suzuki.riaru.n2@elms.hokudai.ac.jp

Yoshinori Dobashi
Hokkaido University
Japan
Prometech CG Research
Japan
doba@ist.hokudai.ac.jp

Abstract

Translucent objects exhibit distinct appearances due to subsurface scattering, but rendering them realistically often involves time-consuming parameter adjustments. This paper proposes an efficient visualization method, using Principal Component Analysis (PCA) and Radial Basis Function (RBF), to allow users to explore subsurface scattering parameters interactively, enabling quicker computation and optimal settings.

CCS Concepts

• Computing methodologies → Rendering.

ACM Reference Format:

Riel Suzuki and Yoshinori Dobashi. 2024. Efficient visualization of appearance space of translucent objects using differential rendering. In *SIGGRAPH Asia 2024 Posters (SA Posters '24)*, December 03–06, 2024, Tokyo, Japan. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3681756.3697925>

1 Introduction

Transparent objects have a unique appearance due to subsurface scattering, and creating realistic images of them requires advanced computer graphics techniques. In movie and video game production, artists often need to adjust parameters repeatedly to achieve specific appearances. However, the nonlinear relationship between parameters and images, along with high computational costs, makes this process difficult. Inverse problem approaches [Dobashi et al. 2012][Nimier-David et al. 2019] can address this, but they require a target image with the desired appearance, which is often unavailable.

In this paper, we propose an alternative solution for parameter exploration in transparent objects by introducing a method to visualize the relationship between parameters and images. Our approach samples a wide range of subsurface scattering parameters and precomputes a database of rendered images. To enhance efficiency, we implement an adaptive sampling scheme using differential images combined with Poisson disk sampling. We then apply Principal Component Analysis (PCA) to project the images

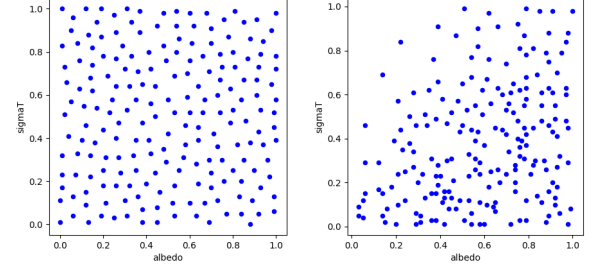


Figure 1: Uniform sampling (left) vs adaptive sampling with differential images (right). Dense samples are generated around the bottom right corner where significant image changes are expected.

into a lower-dimensional space, reducing storage costs and accelerating interpolation. Radial Basis Functions (RBFs) are used for interpolation, allowing users to quickly generate images for various parameters and efficiently identify optimal values to achieve the desired appearance.

2 Proposed method

Among the parameters for the subsurface scattering, we focus on two material parameters: albedo ($\alpha_R, \alpha_G, \alpha_B$) and extinction coefficient σ_t . For albedo, we use three components corresponding to the RGB color channels.

2.1 Construction of Image Database

We represent each parameter set as a four-dimensional vector and use Poisson disk sampling with differential images for adaptive sampling. Each set is assigned a unique sampling radius r based on the average pixel intensity of its differential image, and any subsequent samples within radius r are rejected. This approach intensifies sampling in areas with significant image changes. To reduce the high computational cost of differential image computation, they are calculated only for a small number of initial samples, then interpolated for subsequent ones. The user specifies the number of images in the database, which are generated using the physically-based renderer Mitsuba3 [Jakob et al. 2022]. Fig. 1 compares different sampling schemes for the parameter space.

To reduce storage costs, PCA is applied to generate basis images, represented as $B_j^\lambda(p)$ ($j = 0, \dots, N_B - 1$), where N_B is the number of the basis images. Let us denote each of the RGB channels of the i -th image in the database by $I^\lambda(p, \mathbf{q}_i^\lambda)$ ($\lambda = R, G, B$), where p is a

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SA Posters '24, December 03–06, 2024, Tokyo, Japan

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1138-1/24/12

<https://doi.org/10.1145/3681756.3697925>

pixel and $\mathbf{q}_i^\lambda = (\alpha_\lambda, \sigma_t)$. Using the basis images, the images in the database are approximately represented by the following equation:

$$I_i^\lambda(p, \mathbf{q}_i^\lambda) \approx \sum_{j=0}^{N_B-1} c_j(\mathbf{q}_i^\lambda) B_j^\lambda(p), \quad (1)$$

where $c_j(\mathbf{q}_i^\lambda)$ is the weight of the j -th basis image and is calculated by the following equation.

$$c_j(\mathbf{q}_i^\lambda) = \sum_p I_i^\lambda(p, \mathbf{q}_i^\lambda) B_j^\lambda(p). \quad (2)$$

Eq. 2 is calculated and stored during the creation of the database. Thus, only the basis images and the weight corresponding to each image are stored, significantly reducing storage costs.

2.2 Efficient Rendering by Interpolation

An image with an arbitrary parameter set is efficiently generated by interpolation using radial basis functions (RBFs). Our interpolation method works on each of the RGB color channels separately.

We represent the image with parameter set $\mathbf{q}^\lambda$ with a linear combination of the basis images, that is,

$$I^\lambda(p, \mathbf{q}^\lambda) = \sum_{j=0}^{N_B-1} c_j(\mathbf{q}^\lambda) B_j^\lambda(p). \quad (3)$$

$c_j(\mathbf{q}^\lambda)$ is interpolated using RBFs from the weights in the database (Eq. 2):

$$c_j(\mathbf{q}^\lambda) = \sum_{i=0}^{N-1} \xi_{ij}^\lambda \phi(\|\mathbf{q}^\lambda - \mathbf{q}_i^\lambda\|), \quad (4)$$

where $\phi(d)$ is the RBF, and ξ_{ij}^λ is its coefficient. ξ_{ij}^λ is obtained by solving the following linear system of equations:

$$c_j(\mathbf{q}_k^\lambda) = \sum_{i=0}^{N-1} \xi_{ij}^\lambda \phi(\|\mathbf{q}_i^\lambda - \mathbf{q}_k^\lambda\|) \quad (k = 0, \dots, N-1) \quad (5)$$

Since ξ_{ij}^λ depends only on the parameters used to generate the database, there is no need to solve the equations at runtime. Thus, interpolation can be performed very quickly through Eqs. 3 and 5.

3 Results

In this section, we present examples demonstrating the proposed method's usefulness. The database contains 1,000 images, each with a resolution of 256×256 . We used a desktop PC with an Intel Core i9-10980XE CPU (3.00GHz) and 256GB of RAM. PCA applied to the images revealed that nine basis images (three per RGB channel) achieved a 99.5% contribution ratio, and image interpolation was performed using these. Creating the database took approximately 6 hours. Fig. 2 shows our visualization system, where images are arranged in 2D screen space using Multidimensional Scaling (MDS). The four-dimensional parameter values for each image were used to compute Euclidean distances, forming an $N \times N$ distance matrix, with MDS determining each image's position on the screen.

The top left image of Fig. 2 shows the system's initial view, displaying all images in the database. Users can zoom in for detailed views (top center). Clicking on the screen brings up an image corresponding to the click location (top right), which is either the

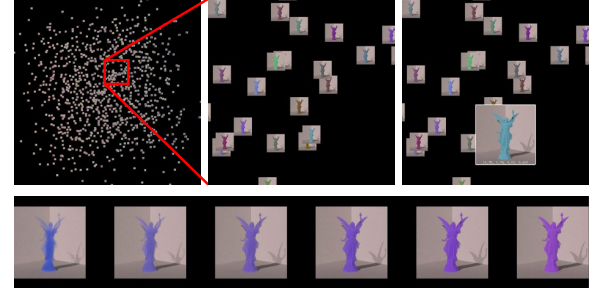


Figure 2: Our visualization system.

closest database image or an interpolated one, based on user preference. The bottom images demonstrate how an object's appearance changes smoothly through interpolation. The images at the left and right ends are from the database, while the four images in between are generated by interpolation.

We investigated the speed and accuracy of the image interpolation using PCA and RBF functions. Parameters were randomly sampled and the corresponding images were generated with the proposed method. We also created reference images using the physically-based path tracer Mitsuba3 [Jakob et al. 2022]. The root mean square error (RMSE) between these images was calculated, resulting in a value of 8.4. The maximum pixel value of the images was 255, and the RMSE was calculated excluding pixels corresponding to the background. The time taken to generate an interpolated image after specifying the parameter values was 0.017 seconds. These experiments indicate that image generation by our method is fast and accurate.

4 Conclusion and Future Work

In this paper, we proposed an efficient method for visualizing the appearance space of translucent objects. By using Poisson disk sampling with differential images, we enabled adaptive sampling in areas with significant image changes. Integrating PCA and RBF allowed for accurate, fast interpolation while reducing image database storage costs.

In the future, we aim to extend the system to include phase functions and specular reflection, and to incorporate an inverse function, enabling users to efficiently find parameters that produce the desired appearance.

Acknowledgments

This work was supported by JSPS KAKENHI Grant Numbers JP22H03710, JP21H03571, and JP20H05954.

References

- Yoshinori Dobashi, Wataru Iwasaki, Ayumi Ono, Tsuyoshi Yamamoto, Yonghao Yue, and Tomoyuki Nishita. 2012. An Inverse Problem Approach for Automatically Adjusting the Parameters for Rendering Clouds Using Photographs. *ACM Trans. Graph.* 31, 6, Article 145 (nov 2012), 10 pages. <https://doi.org/10.1145/2366145.2366164>
- Wenzel Jakob, Sébastien Speierer, Nicolas Roussel, Merlin Nimier-David, Delio Vicini, Tizian Zeltner, Baptiste Nicolet, Miguel Crespo, Vincent Leroy, and Ziyi Zhang. 2022. *Mitsuba 3 renderer*. <https://mitsuba-renderer.org>.
- Merlin Nimier-David, Delio Vicini, Tizian Zeltner, and Wenzel Jakob. 2019. Mitsuba 2: A Retargetable Forward and Inverse Renderer. *ACM Trans. Graph.* 38, 6, Article 203 (nov 2019), 17 pages. <https://doi.org/10.1145/3355089.3356498>